

PHASE 2 BACKEND FOUNDATION REPORT

DWELLERS

Find. Buy. Build.

Delivery record for Phase 2 of the Dwellers platform: a 41-model PostgreSQL-ready domain schema, the nationwide Ghana location and service-area architecture, nine API endpoint groups behind the hardened request pipeline, and a 106-check verification suite proven against a clean database rebuild.

DELIVERY STATUS — COMPLETE

Gate conditions resolved 10 / 10

Verification 106 + 48 checks, all green

Date September 2026

Prepared by Dwellers Engineering

DWELLERS · GHANA · GH¢

Table of Contents

1. Executive Summary	1
2. Scope, Requirements and Method	2
3. Gate Condition Remediation	3
4. Domain Schema Architecture	4
5. Locations, Service Areas and Search	6
6. API Surface and Request Pipeline	7
7. Money Policy and Ghana Localisation	8
8. Seed Data and Demo Safety	9
9. Verification and Clean-Migration Proof	10
10. Documentation and Repository State	11
11. Deferred Scope and Phase 3 Readiness	12

1. Executive Summary

41 Domain models in the Prisma schema	10 / 10 Phase 1 gate conditions resolved	106 + 48 Verification checks, all passing	9 Live API endpoint groups
--	---	--	--------------------------------------

Phase 2 of the Dwellers platform set out to convert the Phase 1 foundation into a working backend: a complete database domain model, real modules with validation and services, a live API surface, seed data for the Ghanaian market, and a verification suite that proves all of it. This report records what was delivered, how each deliverable was verified, and where the deliberate boundaries of the phase were drawn. The work was performed against a 45-part requirements brief supplied by the project owner, and every part of that brief is accounted for in this document.

Three results define the phase. First, the data layer: a 41-model Prisma schema covering identity, businesses, provider profiles, the full nationwide Ghana location hierarchy, service areas, marketplace taxonomy, listings, job requests, quotations, projects, messaging, trust and commerce, applied to the database as migration 20260912025110_init_phase2_domain from a clean state. Second, the API layer: twenty route files across nine endpoint groups, all flowing through the single hardened handler factory that Phase 1 established, with a generic ownership guard that makes insecure direct object references structurally difficult. Third, the proof layer: a 106-check verification suite that rebuilds an isolated database from migrations on every run and exercises every entity, every ownership boundary and the money policy, alongside the original 48-check foundation suite which still passes untouched.

Equally important is what the phase resolved. All ten conditions attached to the Phase 1 gate review were closed at the start of the phase, in commit 21aa5e6: the sixteen TypeScript errors are fixed and build-time type checking is now enforced rather than ignored, the real .env file is untracked, Prisma query logging is environment-scoped, unknown API paths return the JSON error envelope, the Ghana phone regex is tightened, SVG uploads are blocked, storage streaming is typed correctly, the rate limiter evicts under pressure, and proxy trust is opt-in and configurable. The phase closed with a clean git tree, three conventional commits, and no outstanding gate conditions.

Dimension	Result	Evidence
Gate conditions	10 / 10 resolved	Commit 21aa5e6, Chapter 3
Domain schema	41 models, migration applied from clean state	Commit 2e4928e, Chapter 4
Modules and APIs	9 endpoint groups, 20 route files	Commit 4ee9a1f, Chapter 6
Phase 2 verification	106 / 106 on isolated rebuilt database	bun run verify:phase2, Chapter 9
Foundation verification	48 / 48 unchanged and passing	bun run verify
Static analysis	ESLint 0 problems; tsc --noEmit passes	ignoreBuildErrors disabled
Clean rebuild	Migrate deploy + seed + healthy app	Chapter 9
Documentation	DATABASE.md (17 sections); README and ARCHITECTURE updated	Chapter 10

Table 1.1 - Phase 2 delivery summary

2. Scope, Requirements and Method

The phase was governed by a 45-part requirements brief from the project owner. The brief was explicit about sequencing: remediate the Phase 1 gate conditions first, then build the schema, then the API, then seed and tests, then documentation and final verification. It was equally explicit about what Phase 2 should not do. Authentication endpoints and session issuance belong to Phase 3, so authenticated HTTP flows are proven at the service layer with 401s enforced at the API boundary. Quotation, order and payment APIs belong to later phases, so those endpoints are absent even though their models and money policy are ready. Search stays a simple contains query until PostgreSQL arrives. These boundaries were respected throughout, and Chapter 11 records them as deliberate deferrals rather than gaps.

The method followed three principles. Build on Phase 1 rather than rebuilding it: every convention established in the foundation - the module charters, the handler factory, the RBAC engine, the envelope contract - was reused as-is. Prove the database from nothing: the migration was applied to a clean database, not patched on top of development state, and the verification suite rebuilds its own isolated database from migrations on every run so the tests measure the migration path, not the developer's machine. Verify behaviour, not intention: every claim in this report traces to a commit, a script output, a live HTTP response or a browser session.

Requirement parts	Deliverable	Commit
Part 1 (remediation)	All ten gate conditions resolved; type safety enforced at build	21aa5e6
Parts 2-30 (schema)	41-model domain with clean migration	2e4928e
Parts 32-36 (APIs)	Modules, services, 9 endpoint groups, ownership guard	4ee9a1f
Part 37-39 (tests)	106-check suite on isolated database; clean-migration proof	4ee9a1f
Part 40 (seed)	Idempotent nationwide seed with marked demo data	4ee9a1f
Parts 41-42 (docs)	DATABASE.md; README and ARCHITECTURE updates	4ee9a1f
Parts 44-45 (close-out)	Final verification pass and this report	4ee9a1f

Table 2.1 - Requirement parts to deliverables

Version facts for the record: the stack is unchanged from Phase 1 - Next.js 16, TypeScript 5 strict mode, Tailwind 4 with shadcn/ui, Prisma 6, Zod 4, Bun and ESLint 9. What changed in Phase 2 is the depth of the application layer above that stack, and the strictness of the compiler settings beneath it.

3. Gate Condition Remediation

The Phase 1 gate review passed with ten conditions, and the requirements brief made resolving all of them the first task of Phase 2. The conditions fell into two groups: one high-severity item concerning build-time type safety, and nine lower-severity hardening items spanning repository hygiene, error surfaces, logging, upload policy, storage typing, rate limiting and proxy trust. All ten were resolved in commit 21aa5e6 before any Phase 2 feature work began, and none required architectural change - which was exactly the gate review's prediction.

The high-severity condition deserved its severity. The TypeScript build had been inherited from the scaffold with `ignoreBuildErrors` enabled, and behind that flag `tsc` reported roughly sixteen errors: missing type imports for `Role` and `LogEventName` in the core authorization and audit modules, a Zod v4 refine signature change, a readonly modifier conflict in the error type, a logger overload spread, and generic variance in the handler factory, plus scaffold noise from the examples and skills directories. All sixteen were fixed properly - the two missing type imports being the most important, since they sat in the security-critical path - the unrelated directories were excluded from `tsconfig`, `noImplicitAny` was raised to `true`, and the `ignore` flag was removed so type checking now gates every future build alongside `reactStrictMode`.

Sev.	Condition from gate review	Resolution in commit 21aa5e6
HIGH	16 tsc errors masked by ignoreBuildErrors	All fixed; tsconfig excludes examples, skills and tool dirs; noImplicitAny true; ignoreBuildErrors false; reactStrictMode true
MED	Real .env tracked in git	git rm --cached .env; .env.example remains tracked
MED	Unknown /api/* paths returned HTML 404	JSON 404 catch-all route at src/app/api/[...path]/route.ts
MED	Prisma query logging always on	Query logging dev-only; warn and error levels in production
LOW	Phone regex accepted 012345678	Tightened to /^(?:\+2330)[235]\d{8}\$/
LOW	SVG upload XSS surface	SVG removed from all allow-lists; explicit BLOCKED_MIME defence
LOW	Storage stream type assertion	Readable.toWeb conversion; stream() added to StorageProvider interface
LOW	Rate limiter eviction cap	Expired-entries sweep first, then FIFO eviction at the 10,000-bucket cap
LOW	Blind trust in XFF header	Opt-in TRUST_PROXY_ENABLED and TRUSTED_PROXY_HOPS wired into getClientIp and audit context
LOW	Configuration leftovers	Scaffold directories excluded; strict flags raised

Table 3.1 - Gate conditions and resolutions

Two resolutions are worth a sentence more. The storage fix replaced a type assertion with a real `Readable.toWeb` conversion and promoted `stream()` into the `StorageProvider` interface, so cloud drivers in later phases must implement streaming rather than silently buffering. The proxy trust fix turned an implicit assumption into an explicit, documented, opt-in configuration: client IP extraction now honours X-Forwarded-For only when `TRUST_PROXY_ENABLED` is set, and only through the configured number of trusted hops, which keeps audit attribution honest behind a load balancer without exposing the application to header spoofing when deployed directly.

4. Domain Schema Architecture

The schema is the centre of mass of Phase 2: forty-one models in a single Prisma schema file, migrated to SQLite as a 1,125-line SQL migration applied from a clean database. The model set was designed top-down from the marketplace's core question - how does a customer in any Ghanaian town find, evaluate, transact with and stay in contact with a builder, artisan or supplier - and bottom-up from the conventions the platform committed to in Phase 1: integer pesewas for money, integer thousandths for quantities because SQLite has no Decimal type, string status enums validated at the boundary, soft-delete on users, and index-backed lookups on every path the API will walk.

Domain group	Models	Purpose
Identity	User, PersonalProfile	Phone-first accounts, verification state, soft delete
Businesses	Business, BusinessMember	Registered companies with owner-plus-team membership
Providers	ProviderProfile	Individual artisans and business providers; profession, availability, rating summary, completed jobs, response rate
Locations	Region, District, Town, CommunityArea	Nationwide Ghana hierarchy with latitude and longitude
Service areas	ProviderServiceArea, BusinessServiceArea, ServiceServiceArea	Where each actor and listing actually works - reverse-lookup ready
Taxonomy	Category, MeasurementUnit	Hierarchical category tree; data-driven units
Listings	Service, Product, ProductImage, Equipment, EquipmentImage	Six service pricing models; product and equipment catalogues with images
RFQ	JobRequest, JobRequestAttachment	Full-lifecycle request for quotation with attachments and coordinates
Quotations	Quote, QuoteItem	Server-calculated totals; quantity in integer thousandths
Projects	Project, ProjectTask, ProjectMilestone	Engagement tracking after a quote is accepted
Messaging	Conversation, ConversationParticipant, Message, MessageRead, MessageAttachment	Per-request conversations with read receipts
Trust	Notification, Review, Verification	Verified-engagement reviews ready; verification workflow
Portfolio	PortfolioItem, PortfolioImage, Favorite	Work history galleries; polymorphic saved items
Commerce	Order, OrderItem, Payment	Historical price snapshots; gateway-agnostic payments
Audit	AuditLog	Append-only trail carried over from Phase 1

Table 4.1 - Domain groups and models

Three design decisions deserve explanation because later phases depend on them. Provider profiles deliberately unify individual artisans and business-backed providers behind one searchable shape, so discovery does not need to know which legal form stands behind a profile. Service areas are modelled as explicit join tables rather than free-text regions, which is what turns the business question 'which plumbers serve Nsawam?' into an indexed reverse lookup instead of a table scan. And order items store historical price snapshots at purchase time, so a later price change on a product never rewrites financial history.

PostgreSQL readiness was re-verified against the full schema, not just spot-checked: no native SQLite types, no Prisma enums that would need mapping, no JSON columns, and a single portable raw query in the health check. The migration from SQLite to PostgreSQL later in the project is a procedure, documented in DATABASE.md, not a rewrite.

5. Locations, Service Areas and Search

Ghanaian construction hiring is geographic first: a customer in Nsawam does not want the best plumber in Ghana, they want the best plumber who will actually come to Nsawam. Phase 2 encodes that reality directly. The location hierarchy runs Region to District to Town to CommunityArea, all sixteen regions seeded with real districts, towns and communities, and every town and community carrying coordinates where available so distance-aware features can be layered on later without re-modeling.

On top of that hierarchy sit the three service-area join tables from Chapter 4. A provider declares the towns they serve, a business declares the towns it serves, and a specific service listing can narrow further to the towns it covers. All three tables are indexed for the reverse direction - given a town, find everyone who serves it - because that is the direction the product always asks. The live verification in Chapter 9 exercises exactly this: a search for plumbers serving Nsawam returns the seeded provider Kwame Darko, whose service areas include Nsawam, Adoagyiri, Suhum and Koforidua, demonstrating the join path from town through service area to provider profile to user.

Text search is deliberately plain for now. Listing and provider name searches use case-insensitive contains queries, which are correct and portable on SQLite but will not scale to fuzzy or ranked matching. The requirements acknowledged this: when the database moves to PostgreSQL, the search layer graduates to citext columns and pg_trgm indexes with no schema change required, and DATABASE.md notes the planned upgrade path. What matters at this stage is that every search entry point already funnels through the same service functions, so the upgrade is an implementation swap behind a stable interface rather than a rewrite across route files.

Entity	Count	Notes
Regions	16	All Ghanaian regions including the six newer ones
Districts	54	Sample districts covering every region
Towns	54	Regional capitals plus key towns; Nsawam, Adoagyiri and Suhum included
Communities	23	Neighbourhood-level areas, e.g. Asokwa, Ahodwo, Bantama in Kumasi
Measurement units	12	Data-driven units for service and product quantities

Table 5.1 - Seeded geographic reference data

6. API Surface and Request Pipeline

Phase 2 turned the handler factory from a convention into the only door. Twenty route files across nine endpoint groups all flow through the same pipeline - rate limiting, authentication, role-based authorization, Zod validation, the standard response envelope, and structured logging with audit events - in that fixed order. The factory was upgraded in this phase to be schema-type-driven: because request body and query schemas are passed as typed generics, route handlers receive fully typed inputs without casts, which removes an entire class of quiet type lies that loose handlers permit. Named rate-limit presets replace ad-hoc limits, and every domain event that matters writes an audit action.

Ownership enforcement moved from good intentions to a structural guarantee. The new `src/lib/auth/ownership.ts` implements a load-verify-act pattern: the service loads the target row, the guard verifies the caller's relationship to it, and only then does the action run. Missing rows produce 404 and forbidden rows produce 403, so callers cannot distinguish ownership from existence, and staff roles receive a controlled override. The guard also maps indirect ownership - a business owner acting on a business-owned listing, for example - through configurable owner-field paths, which keeps the check in one place instead of scattering role logic across every endpoint.

Group	Routes	Access model
users	<code>/api/users, /api/users/me</code>	Self-service profile; staff user list
businesses	<code>/api/businesses (+[id])</code>	Owner and member management
providers	<code>/api/providers (+[id])</code>	Provider profile CRUD and discovery
categories	<code>/api/categories (+[id])</code>	Public read; admin-managed tree
locations	<code>/api/locations</code>	Public browse of the Ghana hierarchy
services	<code>/api/services (+[id])</code>	Owner-scoped listing management
products	<code>/api/products (+[id])</code>	Owner-scoped listing management
equipment	<code>/api/equipment (+[id])</code>	Owner-scoped listing management
job-requests	<code>/api/job-requests (+[id])</code>	Customer-owned RFQ lifecycle with attachments
health, 404	<code>/api/health, /api/[...path]</code>	DB-connected health; JSON envelope fallback

Table 6.1 - Endpoint groups

The contract at the edges is complete: pagination is clamped and shaped uniformly, errors leave only through the typed envelope, and unknown `/api/*` paths now fall through to the JSON 404 catch-all instead of an HTML page - the last HTML leaking through the API surface, closed as part of gate remediation.

Because authentication endpoints are Phase 3 scope, protected routes currently enforce 401 on anonymous calls, and the authenticated behaviours are proven at the service layer by the verification suite acting as named users. The seam where NextAuth will plug in was left prepared, not improvised.

7. Money Policy and Ghana Localisation

Money errors are the most expensive class of bug a marketplace can ship, so the money policy was written down as code in `finance.ts` and tested as a first-class subject in the verification suite. All amounts are integers in pesewas; all quantities are integers in thousandths, because SQLite lacks a `Decimal` type and floats are forbidden in financial arithmetic. Rounding is `ROUND_HALF_UP` by policy, percentage calculations route through `percentOf`, and the allocation of a total across line items uses the largest-remainder method so that split amounts always re-sum to the original total - the property that keeps invoices and later payment splits reconcilable.

The verification suite caught a real money bug during the phase: the line-total computation divided quantities incorrectly, an error that produced plausible-looking cents-level drift rather than an exception. The fix landed with a regression check, and `lineTotalAmount` now documents its thousandths contract at the type level. That single catch justifies the policy of testing money arithmetic on an isolated database with deliberate edge inputs rather than trusting implementation reviews.

Rule	Definition	Why
Currency unit	Integer pesewas everywhere; cedi formatting at the edge	No float drift; GHc display is a view concern
Quantity unit	Integer thousandths (<code>quantityMilli</code>)	SQLite has no <code>Decimal</code> ; 3 decimal places cover trade quantities
Rounding	<code>ROUND_HALF_UP</code> via <code>roundHalfUp</code>	Predictable, documented, matches commercial expectation
Allocation	<code>allocateAmount</code> with largest remainder	Split totals always re-sum exactly
Line totals	<code>lineTotalAmount</code> from unit price and milli quantity	Single source of truth; regression-tested
Phone validation	<code>/^(?:\+233 0)[235]\d{8}\$/</code> with normalisation	Tightened per gate review; rejects malformed numbers
Locale data	16 regions, districts, towns, communities seeded	Product works for the whole country, not only Accra

Table 7.1 - Money and localisation rules

Localisation is treated as data, not as decoration. The same seed that powers development powers the product's understanding of Ghana: regions including the six newer ones, real district and town names, community-level areas inside major cities, and +233 phone handling that stores a single canonical form.

When Phase 3 adds registration, new users will be validated against exactly this reference data from day one.

8. Seed Data and Demo Safety

A marketplace with an empty database cannot be developed against, demoed, or tested end-to-end, so Part 40 of the brief required an idempotent seed. `scripts/seed.ts` upserts the nationwide Ghana reference data, the marketplace taxonomy, and a small cast of demo actors: four provider profiles across trades, three businesses, five services, three products, two equipment listings and one demo job request, belonging to ten demo users. Idempotence means the seed can run repeatedly - in fresh environments, in CI, after a reset - without duplicating rows or drifting.

Demo safety was designed in rather than added on. Every seeded user carries `isSeedData` set to true, uses an address under the reserved `demo.dwellers.test` domain, and is permanently barred from VERIFICATION status, so demo accounts can never be confused with real customers or promoted into trusted-market privileges. This matters for the trust system specifically: reviews and verification gating are core to the product's promise, and the seed is structured so that demo data can exercise every flow without contaminating the trust signals that real users will rely on.

Seeded set	Volume	Shape
Ghana geography	16 / 54 / 54 / 23	Regions, districts, towns, communities with coordinates
Category tree	hierarchical	Parent-child categories for all listing types
Measurement units	12	Data-driven units for quantities
Demo users	10	<code>isSeedData</code> true; <code>demo.dwellers.test</code> ; never VERIFIED
Demo marketplace	4 / 3 / 5 / 3 / 2	Providers, businesses, services, products, equipment
Demo RFQ	1	Job request exercising the lifecycle statuses

Table 8.1 - Seed contents

The seed also serves as executable documentation of intended shapes: a developer reading `seed.ts` sees how service areas attach to providers, how images attach to listings, and which status values a new job request moves through. `DATABASE.md` links the two, so the seed doubles as the canonical example for anyone writing fixtures or new demo scenarios in later phases.

9. Verification and Clean-Migration Proof

106 / 106

Phase 2 checks on an isolated rebuilt database

48 / 48

Foundation checks still passing

23

Verification sections across every domain

0

ESLint problems; tsc clean

The phase's proof load-bearing wall is `scripts/verify-phase2.ts`, run as `bun run verify:phase2`. Its defining property is isolation: on every invocation it rebuilds a disposable database from the migration files, applies the seed, and runs 106 checks against that fresh state, so the suite tests the migration path that production will use, on data that nothing else has touched. The checks are organised into twenty-three sections covering every entity's creation and validation, every relationship direction that matters, and the cross-cutting engines - money policy, phone validation, storage streaming and traversal defence, rate limiter eviction under storm, and trusted-proxy client IP extraction.

The security core of the suite is its IDOR matrix. The checks prove that customers cannot read or mutate other customers' job requests, that providers cannot touch other providers' profiles or quotes, that suppliers cannot alter other suppliers' listings, that a non-participant cannot read a conversation, and that a foreign provider cannot act on a quote they did not author - each denial verified as a 404 or 403 through the ownership guard, never a 500. Alongside these, invalid identifiers land in safe 404s, unauthenticated calls land in 401s, duplicate unique constraints surface as handled P2002 errors, and pagination bounds behave identically at zero, negative and absurd inputs.

The suite earned its keep by catching three real defects during development, all fixed with regression coverage: the money line-total division error described in Chapter 7, a Zod v4 subtlety where `.partial()` was applied to a refined schema and silently lost constraints, and a static-import hoisting bug in the test harness that changed module initialisation order - converted to dynamic imports. Each of these would have been unpleasant to discover in production; the isolated-database design surfaced all three before the phase closed.

Area	Sections	Representative guarantees
Domain entities	1-17	Users, locations, service areas, providers, businesses, listings, RFQs, quotes, projects, messaging, notifications, reviews, verification, orders, payments
Money and locale	18-19	ROUND_HALF_UP, largest-remainder allocation, thousandths line totals; +233 phone validation
Infrastructure	20-22	Storage streaming and path-traversal blocking; rate limiter eviction storm; trusted-proxy IP extraction
Contracts	23	Validation errors and pagination clamping at the boundary

Table 9.1 - Verification sections (*verify:phase2*)

The clean-migration rehearsal required by the brief was executed literally: the development database was deleted, prisma migrate deploy applied 20260912025110 from nothing, the seed ran, and the application came up healthy against the fresh database. Final close-out verification also re-ran the whole battery on the normal tree - ESLint zero problems, tsc --noEmit clean with the ignore flag still off, both suites green - and live HTTP probes confirmed the running app: health with database connected, JSON 404 on unknown API paths, the category tree and location endpoints, the plumbers-in-Nsawam discovery query from Chapter 5, listing endpoints, and 401s on protected routes as designed for this phase. Browser sessions on desktop and mobile rendered cleanly with zero console errors.

10. Documentation and Repository State

A backend of this size is only maintainable if its conventions live in the repository, so the phase delivered DATABASE.md as a new 17-section guide: conventions shared by every model, the full entity map, per-domain walkthroughs from identity through commerce, the authorization model including the ownership guard's 404/403 semantics, and operational procedures for migrations, seeding and the PostgreSQL transition. One section deserves naming - 'How Dwellers finds an artisan' - because it narrates the join path from a customer's town through service areas to provider profiles in plain language for future contributors.

README and ARCHITECTURE were updated to reflect the new reality rather than the aspiration: the architecture guide now documents the module services and their zod schemas, the handler factory's preset and generics, and the ownership layer, while the README records the working scripts a developer actually uses - `verify`, `verify:phase2`, `db:seed`, `db:migrate:deploy`. Documentation drifted not at all from implementation because both were changed in the same commit set.

Commit	Subject	Contents
21aa5e6	fix: resolve Phase 1 gate conditions	Type safety enforced, .env untracked, JSON 404, hardened limiter, proxy trust, storage streaming
2e4928e	feat: add complete Phase 2 domain schema	41 models, 1,125-line migration applied from clean state
4ee9a1f	feat: build Phase 2 backend foundation	Modules and services, 9 API groups, ownership guard, seed, 106-check suite, docs

Table 10.1 - Phase 2 commit register

Repository hygiene is where the phase ends and the gate review's concerns stay closed: the working tree is clean, the three commits are conventional and scoped, .env is untracked while .env.example remains the tracked template, and no artifact of this report generation touched project source. The commit register above is the complete change footprint of Phase 2.

11. Deferred Scope and Phase 3 Readiness

Phase 2 drew three deliberate boundaries, all mandated by the requirements brief and all leaving prepared ground rather than debt. Authentication endpoints and session issuance are Phase 3 scope; the API therefore enforces 401 on protected routes today, and the authenticated behaviours are proven at service level by the verification suite acting as named users. Quotation, order and payment APIs are later-phase scope; their models, statuses and the money policy they will consume are already in place and tested. Search remains a portable contains query until PostgreSQL arrives, with the citext and pg_trgm upgrade path documented. None of these deferrals required a workaround that later phases will need to unwind.

Deferred item	Lands in	State left behind
Auth endpoints and session issuance	Phase 3	401s enforced; NextAuth seam prepared; service-layer flows proven
Quotes, orders, payments APIs	Later phases	Models, snapshots and money policy ready and tested
PostgreSQL migration	Infrastructure step	Portable schema; documented procedure; no blocker sweep items
Fuzzy search (citext, pg_trgm)	Post-PostgreSQL	contains search behind stable service interface

Table 11.1 - Deliberate deferrals

Readiness for the next phase is concrete. The data model is complete enough that Phase 3's registration flows attach to real constraints rather than placeholders; the handler factory's auth step is a standing slot waiting for the session layer to feed it; the RBAC engine from Phase 1 already knows the eight roles and their escalation rules; and the verification pattern is established, so Phase 3 inherits both a blueprint and a bar: new features arrive with their own isolated-database checks or they are not done.

The recommended opening sequence for Phase 3 mirrors this phase's discipline: begin with session issuance and the credential endpoints, wire them into the prepared NextAuth seam, extend the ownership guard's callers to session-backed identities, then build the role-based dashboards on top of the existing API groups, with each step landing in the verification suite. Per the project's standing rule, Phase 3 does not start without the project owner's explicit approval; this report closes Phase 2 and stands ready as the baseline for that decision.

Phase Ledger at the Close of Phase 2

For orientation, the table below places Phase 2 in the context of the project's whole delivery so far, and states the standing position of each stage. It is the single reference a reviewer can use to see, at a glance, what has been built, what has been independently verified, and what is waiting on a decision rather than on engineering.

Stage	Scope	Status and evidence
Phase 1	Architecture, security foundation, RBAC, brand landing	Complete; 4 conventional commits; 48/48 foundation checks
Gate review	Independent adversarial audit of Phase 1	PASS WITH CONDITIONS; 109/109 checks; 10 conditions registered
Phase 2	41-model schema, 9 API groups, seed, 106-check suite	Complete; 3 commits; 10/10 gate conditions resolved
Phase 3	Authentication, registration, role-based dashboards	Not started; awaiting project owner approval

Table 11.2 - Project phase ledger

The Phase 2 gate is closed: schema live, APIs proven, conditions cleared. Phase 3 begins only on the project owner's explicit instruction.